Phase 2: Log Analytics System for an Organization

Sabin Ranabhat

University of the Cumberlands

Algorithms and Data Structures - MSCS-532-B01

Michael Solomon

March 27, 2026

Abstract

Enterprise systems like project management platforms, source code repositories, and cloud infrastructure as well as network services produce high amounts of activity logs in modern organizations. These logs include useful data concerning the activities of employees, the functioning of the system, and their large volumes and also the hetero-structured nature makes their analysis difficult. The design of a Log Analytics System can help aggregate logs of various sources and analyze them in terms of effective data structures developed with Python. The system is based on the use of hash tables to aggregate logs, queues to process chronologically, and priority queues to rank productivity, and hierarchical mappings to investigate the department in a particular order. These data formats allow high-performance processing of large volumes of logs and facilitate analytical operations, including the process of determining productivity trends and usage of the system, and producing insights related to management. The suggested design illustrates how theories derived in the area of data structures and algorithms could be used in solving real-life enterprise analytics issues.

Keywords: log analytics, data structures, organizational analytics, Python, activity logs

1. Partial Implementation Overview

This project implements a Log Analytics System which is aimed at building an architecture with high scalability and modularity based on core data structure to model the real-life enterprise log analysis. The system incorporates various aspects, such as log ingestion, processing, scoring, ranking, and analytics, all arranged around effective use of data structure.

The system uses multiple fundamental data structures:

1.1 Hash Table – Log Storage

A hash table (Python dictionary) at the core of the system is used to store logs grouped by employee ID . This will enable constant-time access in retrieving all logs related to a particular employee. All the logs are added to a list towards each employee, so that there may be an efficient accumulation of activity data over time.

```
self.employee_logs: Dict[str, List[dict]] = {}
```

- Groups logs by employee_id
- Allows fast O(1) insertion and lookup
- Facilitates efficient aggregation of employee activity

This serves as the central repository of all analytics processes.

1.2 Queue (FIFO Processing)

A queue has been established using collections.deque in order to maintain a sequential processing of the logs. It allows log processing in the First-In-First-Out (FIFO) order which is quite similar to the actual log streaming system in which events are processed in the order of their reception. This structure can be used especially to debug, monitor and simulate real-time ingestion pipelines.

```
self.log_queue = deque()
```

- Assures logs are processed in arrival sequence(FIFO)
- Simulates real-world log streaming systems
- Supports sequential log inspection
- Simulation of Real time processing

1.3 Priority Queue (Heap) – Ranking System

A priority queue (heap) is used by using the heapq Python module to rank employees according to productivity. This will enable the easy retrieval of the best performing employees without the necessity to sort the whole databank. The heap makes sure that prioritization of operations is done in an optimum way particularly when handling big data.

```
heapq.heappush(heap, (-score, emp_id))
```

- Effectively identifies the best performing employee
- Maintains ranking in O(n log n)
- Generates Top-N employees by activity score

1.4 Config-Based Scoring System

External JSON file (scores.json) is used to implement the scoring system based on a configuration. This design does not bind business logic or code together and makes it flexible to make scoring rules changeable without changes to the fundamental implementation. The activities are divided into high, medium, neutral and negative, with each having certain values reserved to each activity.

```
{
 "high": ["completed", "deploy", "commit", "merge"],
 "medium": ["created", "started", "pull"],
 "neutral": ["login", "logout"],
 "negative": ["cancelled", "deleted", "reassigned"],

 "values": {
  "high": 5,
  "medium": 2,
  "neutral": 0,
  "negative": -3
 }
}
```

1.5 Normalized scoring model

A normalized scoring system is also implemented in the system, in which the raw scores are changed to the efficiency score (score per log) which is in turn changed to the percentile-based scores ranging from 0 to 100. This makes it fair and scalable particularly when it deals with millions of logs.

2. Demonstration and Testing

2.1 Test Setup

In order to test the system, a synthetic data generator (datagenerator.py) was created to mimic the production of logs of enterprise systems like Jira, GitHub, AWS, and VPN. The generator created a realistc activity allocations among various employees and projects.

We generated synthetic logs using:

```
python3 data_generator.py 10000
```

Sample Log:


```
[
  {
    "timestamp": "2026-03-26 17:27:28",
    "employee_id": "E106",
    "source": "GitHub",
    "activity": "Merge",
    "project": "ProjectA",
    "details": {
      "repo": "api-service",
      "lines_changed": 107
    }
  }
]
```


2.2 System Execution

It was tested using 10,000 generated logs and demonstrated that it works well with large data volumes. The logs were ingested into the system and processed in FIFO order during execution.

```
python3 main.py
```

2.3 Sample FIFO Output

A sample of processed logs is shown below:

```
sadhanakhadka@Mac MSCS532_Project % python3 main.py

=== Processing Logs (FIFO) ===

{'timestamp': '2026-03-26 17:27:28', 'employee_id': 'E106', 'source': 'GitHub', 'activity': 'Merge', 'project': 'ProjectA', 'details': {'repo': 'api-service', 'lines_changed': 107}}
{'timestamp': '2026-03-26 17:29:12', 'employee_id': 'E106', 'source': 'Jira', 'activity': 'Task Started', 'project': 'ProjectC', 'details': {'task_id': 'JIRA-760'}}
{'timestamp': '2026-03-26 17:29:14', 'employee_id': 'E102', 'source': 'Jira', 'activity': 'Task Created', 'project': 'ProjectA', 'details': {'task_id': 'JIRA-296'}}
{'timestamp': '2026-03-26 17:30:01', 'employee_id': 'E102', 'source': 'VPN', 'activity': 'Logout', 'project': None, 'details': {'ip': '192.168.1.52'}}
{'timestamp': '2026-03-26 17:32:08', 'employee_id': 'E112', 'source': 'GitHub', 'activity': 'Merge', 'project': 'ProjectB', 'details': {'repo': 'api-service', 'lines_changed': 471}}
{'timestamp': '2026-03-26 17:33:28', 'employee_id': 'E119', 'source': 'VPN', 'activity': 'Logout', 'project': None, 'details': {'ip': '192.168.1.171'}}
{'timestamp': '2026-03-26 17:39:28', 'employee_id': 'E117', 'source': 'GitHub', 'activity': 'Commit', 'project': 'ProjectB', 'details': {'repo': 'api-service', 'lines_changed': 151}}
{'timestamp': '2026-03-26 17:35:38', 'employee_id': 'E108', 'source': 'GitHub', 'activity': 'Merge', 'project': 'ProjectC', 'details': {'repo': 'api-service', 'lines_changed': 441}}
{'timestamp': '2026-03-26 17:40:32', 'employee_id': 'E112', 'source': 'GitHub', 'activity': 'Merge', 'project': 'ProjectA', 'details': {'repo': 'backend', 'lines_changed': 57}}
{'timestamp': '2026-03-26 17:35:43', 'employee_id': 'E111', 'source': 'Jira', 'activity': 'Task Completed', 'project': 'ProjectA', 'details': {'task_id': 'JIRA-379'}}
```

2.4 Final Ranking Output

Employee scores were then computed and ranked after processing. The final output shows raw scores, efficiency scores, and normalized percentile scores. A sample output is shown below:

```
=== Top Employees (Final Score 0–100) ===

Emp ID | Name             | Department | Logs | Raw Score | Efficiency | Final Score
-------------------------------------------------------------------------------------
E113   | Alexander Harris | Finance    | 503  | 1250      | 2.49       | 100.0
E117   | Lucas Martinez   | Finance    | 536  | 1229      | 2.29       | 95.0
E109   | Daniel Taylor    | Finance    | 491  | 1121      | 2.28       | 90.0
E119   | Henry Clark      | Operations | 509  | 1155      | 2.27       | 85.0
E103   | Michael Brown    | Operations | 491  | 1103      | 2.25       | 80.0
```

This output shows the system is properly ranking employees in terms of normalized performance and not based on raw activity volume. Employees with higher efficiency are ranked higher even when their total number of logs is lower.

2.5 Department Analytics Output

Department level analytics were also calculated aggregating the total logs and total scores:

```
=== Department Analytics ===

Department       | Total Logs | Total Score
---------------------------------------------
Engineering      | 2466       | 5248
Customer Success | 443        | 885
Marketing        | 2520       | 5342
Operations       | 2543       | 5531
Finance          | 2028       | 4694
```

These findings support the fact that the system is able to conduct aggregation and analysis across the organizational units.

3. Implementation Challenges and Solutions

3.1 Fair Scoring Across Large Data

Assuring fairness in scoring across employees with different levels of activity. Initially, when raw scores were used directly for ranking it heavily favored employees with a higher number of logs. To overcome this problem, an efficiency-based scoring method was developed where the raw score is divided by the number of logs. This changes made sure that employees are evaluated based on the quality of their activities rather than just quantity.

Implemented normalization using efficiency:

```
efficiency = raw_score / total_logs
```

Then applied percentile scaling:

```
(rank / total_values) * 100
```

- Fair comparison across employees
- Score bounded between 0–100

3.2 Handling Large Log Volumes Efficiently

Processing large volumes of data was also a problem. When it comes to processing thousands of logs it resulted in repetitions of computation tasks, which cause performance inefficiencies, particularly at the department level of analytics. This was addressed by restructuring the computing process and making sure that the log counts and scores were computed in a more optimized way.

- Dictionary aggregation (O(1))
- Heap-based ranking (O(n log n))
- Deque for FIFO processing

3.3 Department-Level Aggregation Complexity

Performance was impeded by re-calculations of scores of employees.

Optimized structure by:

- Computing logs separately once
- Aggregating scores in batch

3.4 Flexible Scoring Design

Hardcoding scores on activities would have rendered the system to be inflexible and difficult to maintain. This was fixed by moving the scoring logic to a separate JSON-based configuration file, and then letting the scoring logic be updated and customized easily without meta-editing the underlying code.

4. Next Steps

Although the POC shows the applicability of the core data structures and batch-oriented log analytics, the next step will be focused at transforming the system into a more realistic and dynamic processing platform with logs. This is followed by the replacement of the currently used, inactive and batch-driven form of computation with the one that facilitates continuous data input and graduating updates without making the processes less efficient and scalable.

The introduction of a polling-based mechanism of near real-time log processing is, perhaps, one of the most valuable improvements. The system will not need to load the whole data in batch and process it simultaneously, but keep on checking the log file, and process only those entries which are appended to it. These will be done through having a pointer or index to indicate the last position in the file that is being processed. The system will periodically (after some time interval) read new logs added since the last cycle and process them step by step. This method is an emulation of the real life log monitoring systems.

In order to facilitate this streaming like behavior, incremental scoring and state management will be added to the system. With the implementation as it is, the scores of employees are computed again as a process in which each log previously stored is iterated. This will be streamlined in Phase 3 by keeping running aggregates of each employee e.g. cumulative scores and the number of logs. When new logs are fed, the new system will only update these values and not compute them afresh. Such change substantially boosts performance and makes the system effective by scaling as the total amount of logs grows.

Besides incremental scoring, the ranking mechanism will be extended to facilitate dynamic changes to Top-N employees. The system will not have to recalculate the ranking list on a query-by-query basis, but instead the rankings would be dynamic and updated on a timely basis based on the more recent scores. This guarantees that benchmark position of the leaderboard is up to date generating minimal computing costs. The current system of priority queueing can be provided to enable this behavior with ease by reusing the old system or by adding new features to it.

The other intended improvement is the adjustment of the data generation workflow to encourage continuous ingestion. The generator will also be modified to add new logs to the previous dataset instead of replacing the log file with a new one each time it is executed. This enables the analytics system to process and recognize only new records of logs every polling cycle. Alternatively, there can be a delay mechanism which now is inserted into the generator to emulate the arrival of the logs with time passing and this takes the system one step nearer to being real.

Also the scoring model may be extended to include time conscious analysis in which activities done previously will be less important than those done recently. This may be done through techniques like sliding windows or decay functions to enable the system to index the current trends of performances to a greater accuracy. Although not necessary in basic mode, this would provide greater analytical capability to the system.

In terms of design, Phase 3 will also focus on extensibility and modularity, that is, different scoring strategies or ranking can be plugged into the system with minimal modifications. This makes the architecture very flexible and easy to maintain with the introduction of new features.

5. Code Snippets

The core scoring logic:

```
def get_activity_score(self, activity: str) -> int:
    activity = activity.lower()


    for category in ["high",  "medium", "neutral", "negative"]:
        if any(keyword in activity for keyword in self.score_data[category]):
            return self.score_data["values"][category]


    return 0
```

This function dynamically assigns scores based on activity keywords and making the system flexible and extensible.

The efficiency computation:

```
def compute_efficiency(raw_score, total_logs):
    if total_logs == 0:
        return 0
    return raw_score / total_logs
```

This ensures fair comparison across employees with varied workloads.

The percentile scoring mechanism:

```
def percentile_function(values):
    sorted_vals = sorted(values)

    def get_percentile(v):
        rank = sum(1 for x in sorted_vals if x <= v)
        return round((rank / len(sorted_vals)) * 100, 2)

    return get_percentile
```

This converts efficiency scores into a normalized 0–100 scale based on relative ranking.

6. Conclusion

The implementation of Phase 2 is a successful demonstration of the ability to integrate various data structures into a scalable log analytics system. The system uses hash tables, queues, and heaps in combination with an effective scoring system, which helps to process data and evaluate performance efficiently and fairly. Normalization techniques provide scalability and applicability to real world, and hence this implementation forms a solid basis to continue its growth.

7. References

1. Chen, Z., Liu, J., Gu, W., Su, Y., & Lyu, M. R. (2021). *Experience Report: Deep Learning-based System Log Analysis for Anomaly Detection*. https://doi.org/10.48550/arxiv.2107.05908
2. Xu, W., Huang, L., Fox, A., Patterson, D., & Jordan, M. I. (2009). Detecting large-scale system problems by mining console logs. *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, 117–132. https://doi.org/10.1145/1629575.1629587
3. Ali, S., Boufaied, C., Bianculli, D., Branco, P., & Briand, L. (2025). A comprehensive study of machine learning techniques for log-based anomaly detection. *Empirical Software Engineering*, 30(5). https://doi.org/10.1007/s10664-025-10669-3
4. Avello Martínez, R., Fernández-Álvarez, D., & Gómez Rodríguez, V. G. (2023). Moodle logs analytics: An open web application to monitor student activity. *Universidad y Sociedad*, 15(4), 715–721.
5. Hamooni, H., Debnath, B., Xu, J., Zhang, H., Jiang, G., & Mueen, A. (2016). LogMine: Fast pattern recognition for log analytics. *Proceedings of the 25th ACM International Conference on Information and Knowledge Management*, 1573–1582. https://doi.org/10.1145/2983323.2983358
6. GitHub Repository: https://github.com/sranabhat51358/MSCS532_Project.git